

# Aula 3 — Serviços, Cliente/Servidor, Virtualização e Containers

---

## 1. Serviços (o que roda sem você pedir)

---

Na Aula 1 vimos processos: programas em execução. Um **serviço** (Linux chama de *daemon*; Windows chama de *service*) é um tipo especial de processo: ele **fica rodando em segundo plano**, sem interface visível, geralmente iniciado automaticamente quando o computador liga, e continua ativo mesmo sem nenhum usuário logado.

Exemplos do dia a dia: o servidor web que hospeda um site, o servidor SSH que aceita conexões remotas, o antivírus/EDR que monitora o sistema o tempo todo, o serviço que sincroniza a hora do relógio.

**Por que importa para segurança:** serviços são um dos alvos favoritos de um atacante para conseguir **persistência** — ou seja, garantir que o código malicioso continue rodando mesmo depois de reiniciar a máquina, ou mesmo se o usuário original nunca mais logar. Criar um serviço novo malicioso, ou modificar um serviço legítimo para executar algo a mais, é uma técnica muito comum e catalogada no MITRE ATT&CK como **T1543 – Create or Modify System Process** (vamos formalizar essa notação de técnicas no Módulo 13). Por isso, **criação ou alteração de serviço** é um dos eventos que todo SOC monitora de perto (no Windows, isso gera o Event ID 7045 no log de sistema, e o Sysmon também registra criação de processos filhos incomuns associados a esse comportamento — veremos isso com detalhe no módulo de Sysmon).

## 2. Arquitetura Cliente/Servidor

---

A maior parte da comunicação em rede segue este modelo:

- **Cliente:** quem inicia a conexão e pede algo (ex.: seu navegador pedindo uma página).
- **Servidor:** quem fica esperando conexões e responde ao pedido (ex.: o servidor que hospeda o site).

```
Cliente (navegador) --pedido--> Servidor (site)
Cliente (navegador) <--resposta-- Servidor (site)
```

Um servidor **escuta** em uma porta específica (ex.: porta 443 para HTTPS) esperando conexões chegarem — vamos formalizar "porta" e "protocolo" com profundidade no Módulo 03 (Redes). Por agora, o que importa é o papel de cada lado: quem inicia (cliente) e quem espera (servidor).

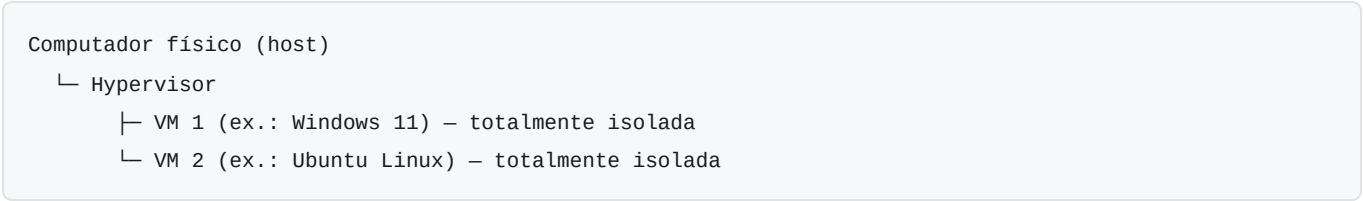
**Por que importa para segurança:** essa distinção é a base de análise de tráfego de rede. Uma das primeiras perguntas em qualquer investigação de rede é: **"quem iniciou essa conexão — o host interno ou algo de fora?"** Uma máquina da sua rede que, de repente, começa a se conectar (como cliente) para um servidor desconhecido na internet, em um horário incomum, é um padrão clássico de comunicação com um servidor de Comando e Controle (**C2**) de um atacante — algo que vamos aprofundar quando estudarmos Network Security e Threat Intelligence.

## 3. Virtualização

---

**Virtualização** é a capacidade de rodar um "computador dentro do computador" — uma **máquina virtual (VM)** é um ambiente completo (sistema operacional próprio, com seus próprios processos, usuários, arquivos) que roda isolado

dentro de um computador físico real (o **host**), usando um software chamado **hypervisor** (ex.: VMware, VirtualBox, Hyper-V, KVM) para dividir os recursos reais (CPU, RAM, disco) entre as VMs.



**Por que importa para segurança:** virtualização é a base de **quase todo laboratório de cibersegurança**, incluindo o que vamos construir aqui neste repositório (pasta `lab/`). Ela permite rodar malware, testar exploits, simular ataques e investigar incidentes **sem risco** para o seu computador real, porque a VM é isolada — se algo der muito errado, você apaga a VM e recomeça. É também a base de muita infraestrutura corporativa (servidores de empresas rodam como VMs, não em hardware dedicado).

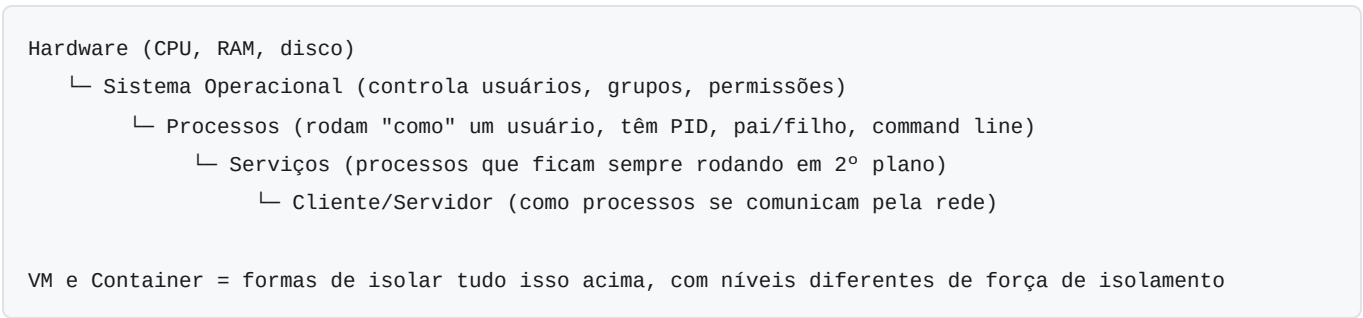
## 4. Containers

**Container** é parecido com uma VM na ideia (isolar uma aplicação), mas funciona de um jeito bem diferente por dentro: em vez de simular um computador inteiro com seu próprio sistema operacional, o container **compartilha o kernel do sistema operacional do host** e isola apenas o que a aplicação precisa (arquivos, bibliotecas, processos). O exemplo mais conhecido é o **Docker**.

|                     | Máquina Virtual                    | Container                               |
|---------------------|------------------------------------|-----------------------------------------|
| Isola               | Hardware completo (via hypervisor) | Processos/arquivos (via kernel do host) |
| Tem seu próprio SO? | Sim, completo                      | Não — usa o kernel do host              |
| Peso/velocidade     | Mais pesado, inicia em minutos     | Leve, inicia em segundos                |
| Isolamento          | Mais forte                         | Mais fraco (compartilha o kernel)       |

**Por que importa para segurança:** como o container compartilha o kernel com o host, o isolamento é **mais fraco** do que numa VM — uma vulnerabilidade grave no container pode, em certos cenários, permitir que o atacante "escape" e afete o host (chamado de *container escape*). Isso faz da segurança de containers uma área própria dentro de Cloud Security/DevSecOps. Não vamos aprofundar isso agora — só é importante que você já saiba diferenciar VM de container quando o assunto aparecer.

## 5. Conectando tudo até aqui (Aulas 1-3)



Com essa base, você já tem vocabulário suficiente para começar a mexer de verdade em um terminal Linux — que é exatamente o próximo módulo.

## 6. Exercício prático

---

Se você tiver um terminal Linux/macOS disponível:

```
# Ver serviços ativos no momento (sistemas com systemd)
systemctl list-units --type=service --state=running | head -15
```

Observe os nomes dos serviços rodando. Não se preocupe em entender cada um agora — o comando `systemctl` será estudado com profundidade no Módulo 01. O objetivo aqui é só visualizar, na prática, o conceito de "processo que fica sempre ativo em segundo plano" que acabamos de explicar.

Se não tiver terminal Linux disponível: no Windows, você pode abrir o **Gerenciador de Tarefas** (Ctrl+Shift+Esc) → aba **Serviços**, e observar a mesma ideia — uma lista de processos rodando em segundo plano, sem janela visível.

## 7. Recapitulando

---

- Serviço = processo que roda em segundo plano continuamente; alvo comum para persistência de malware.
- Cliente = quem inicia a conexão; Servidor = quem espera e responde — essa direção é o primeiro dado que se olha numa investigação de rede.
- VM = isola um computador inteiro via hypervisor; forte isolamento, mais pesada.
- Container = isola só a aplicação, compartilhando o kernel do host; isolamento mais fraco, mais leve.

## Próxima aula

---

Terminal e processos em background — o último passo dos Fundamentos antes de entrarmos, de fato, no Módulo 01: Linux para Blue Team.

## Fontes recomendadas

---

- **Microsoft Learn** — "Services": documentação oficial sobre o modelo de serviços do Windows.  
<https://learn.microsoft.com/windows/win32/services/services>
- **MITRE ATT&CK** — **T1543 (Create or Modify System Process)**: técnica oficial de persistência via serviços.  
<https://attack.mitre.org/techniques/T1543/>
- **Docker Docs** — "What is a container?": explicação oficial do modelo de isolamento de containers.  
<https://docs.docker.com/get-started/docker-overview/>